

Grupo ARCOS

uc3m | Universidad **Carlos III** de Madrid

Tema 3: Fundamentos de la programación en ensamblador (I) **Estructura de Computadores**

Grado en Ingeniería Informática
Grado en Matemática aplicada y Computación
Doble Grado en Ingeniería Informática y Administración de Empresas

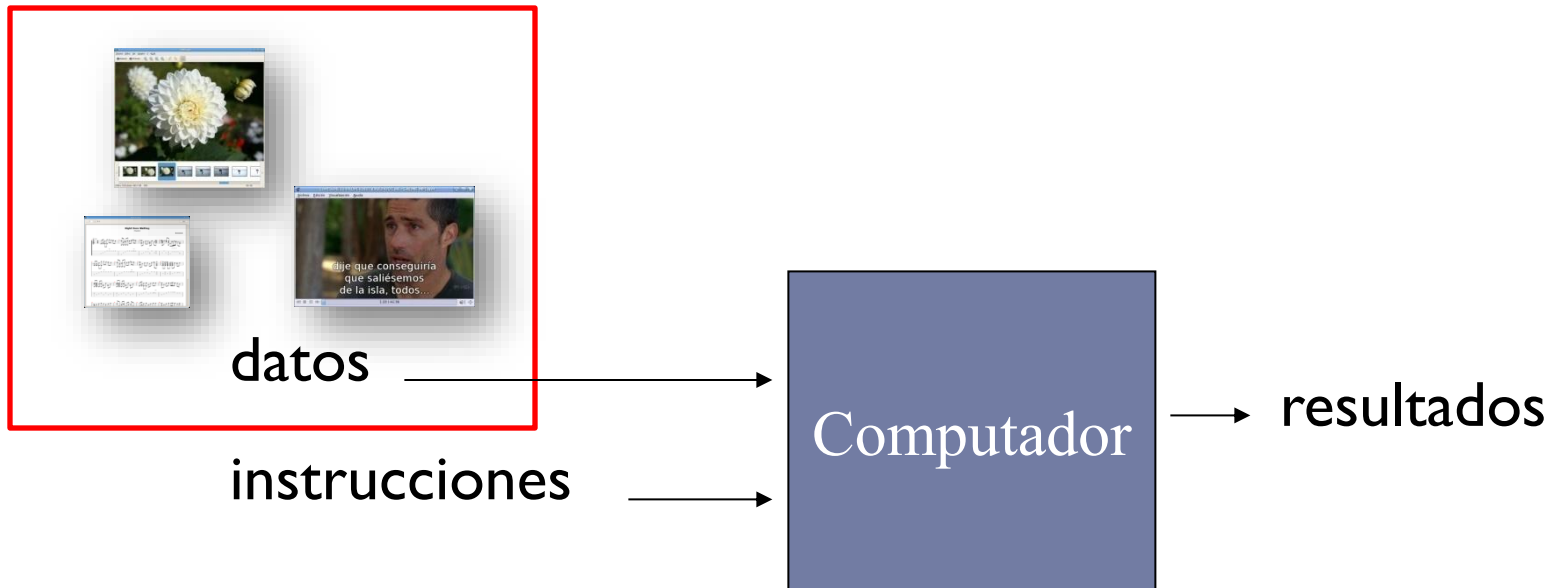


Contenidos

1. Fundamentos básicos de la programación en ensamblador
 1. Motivación y objetivos
 2. Introducción al ensamblador RISC-V 32
2. Ensamblador RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
5. Llamadas a procedimientos y uso de la pila

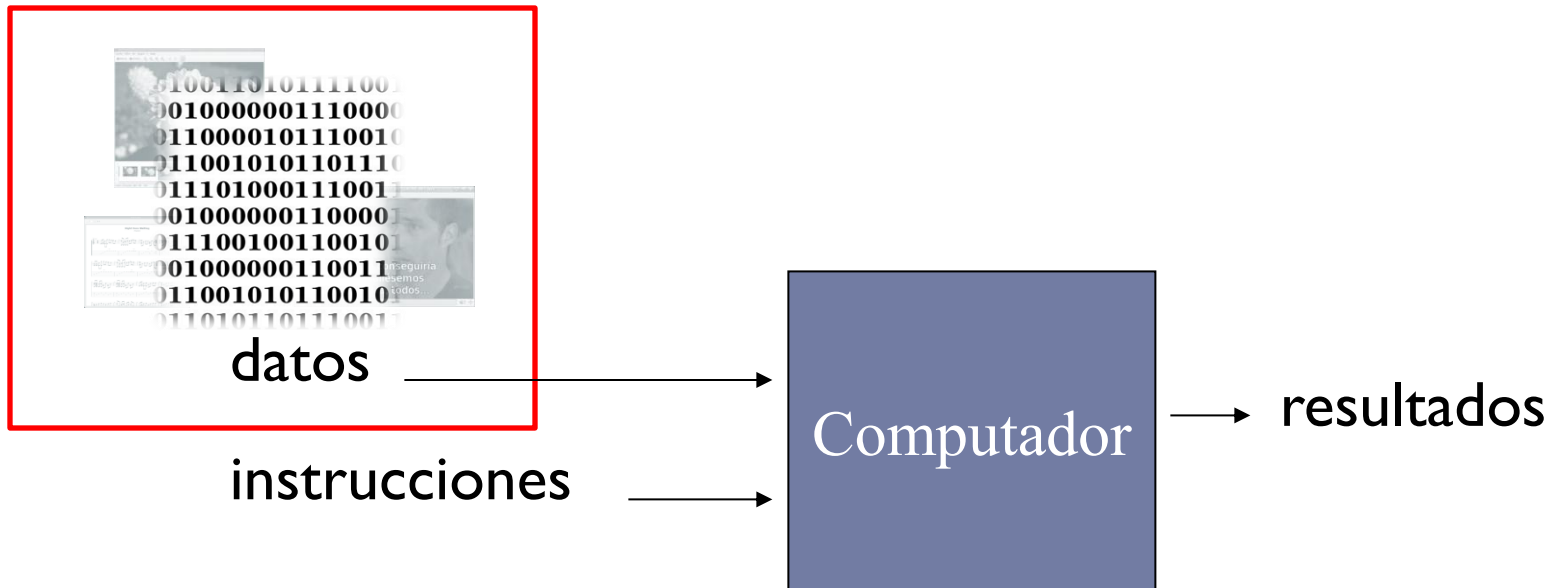
Tipos de información: instrucciones y datos

► Representación de **datos** ...



Tipos de información: instrucciones y datos

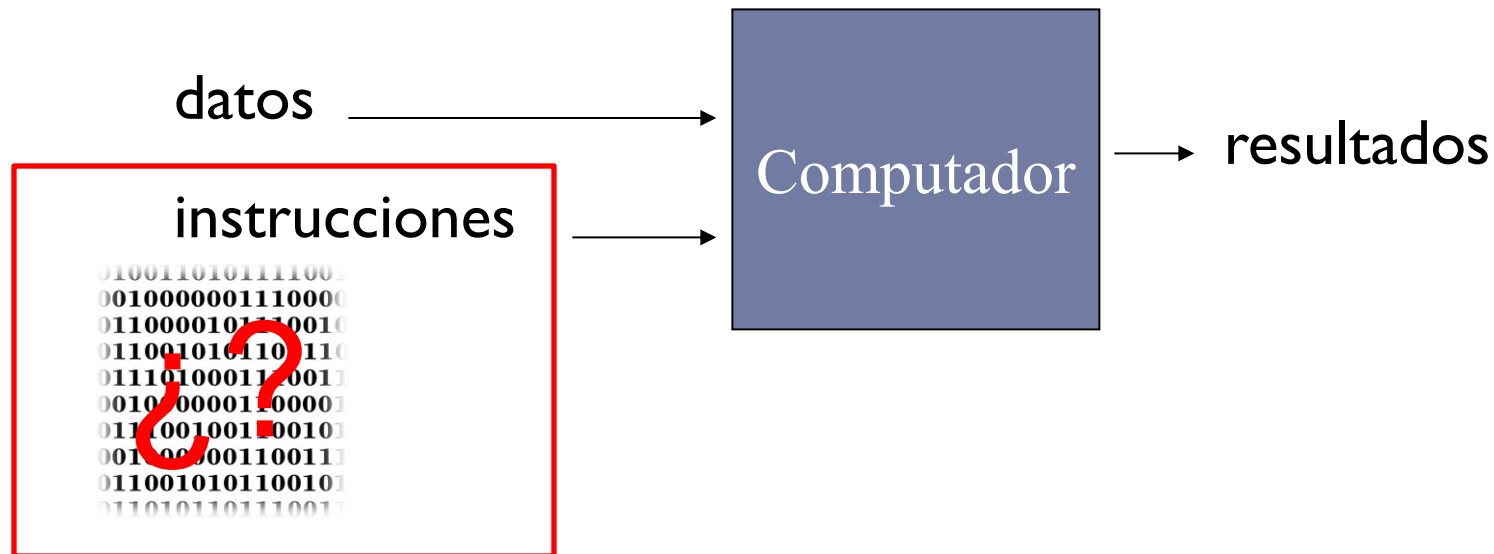
- Representación de **datos** en **binario**.



Tipos de información: instrucciones y datos

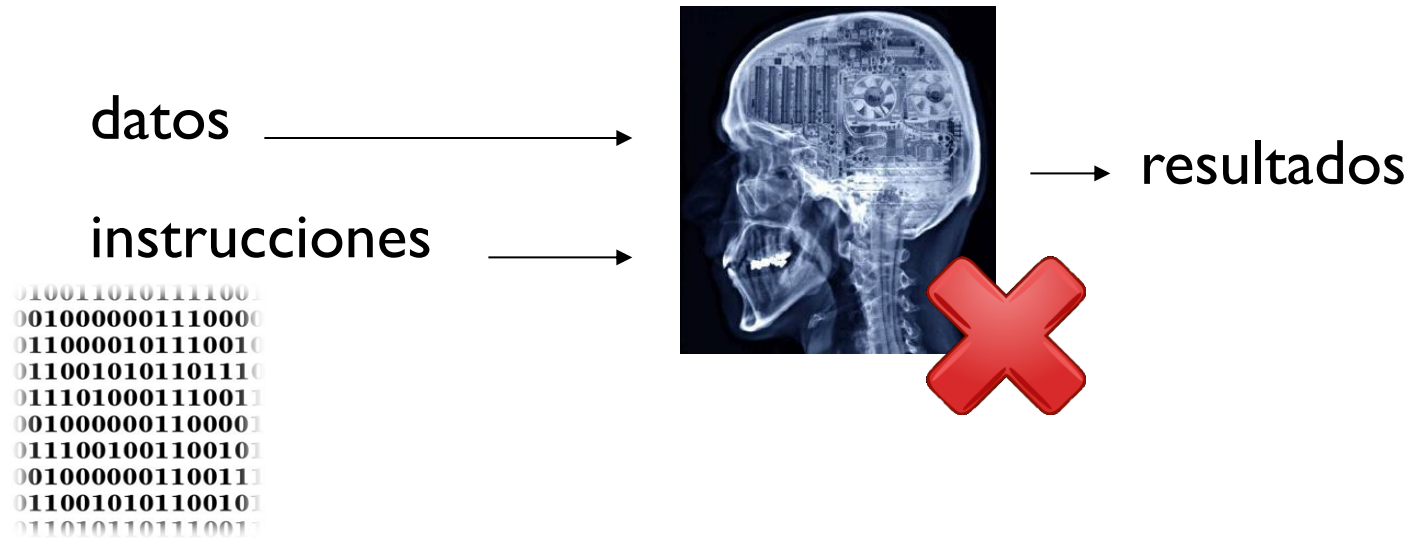
► Instrucciones:

- Cómo son, representación, propiedades y formato, programa, ISA, etc.



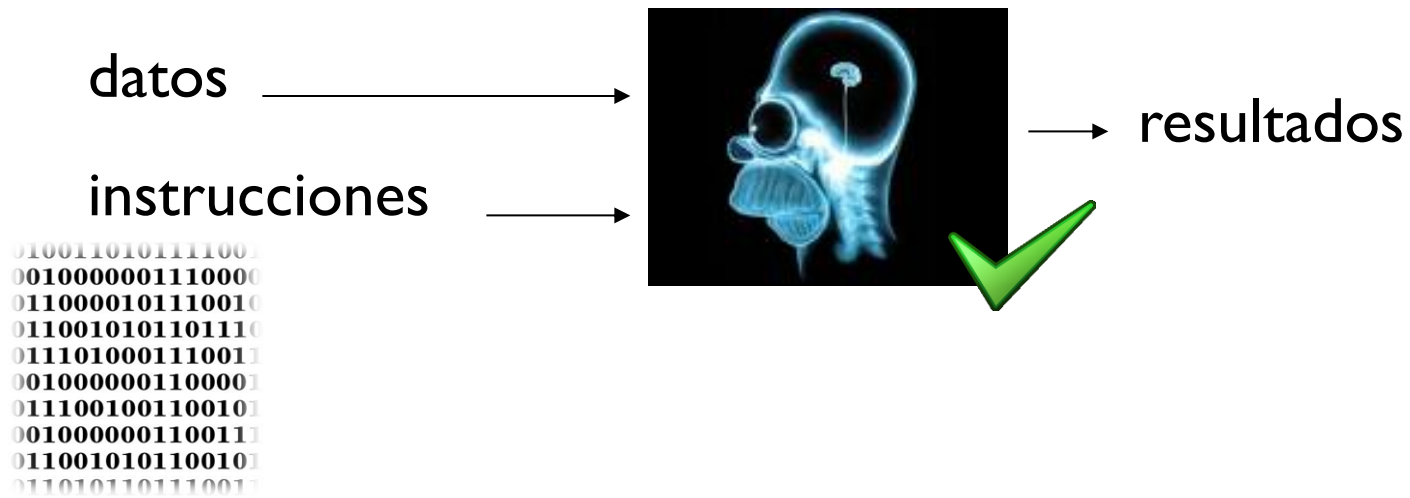
Instrucciones máquina

- ▶ No son instrucciones complejas...



Instrucciones máquina

- ... sino tareas muy básicas...



Instrucciones máquina: definición

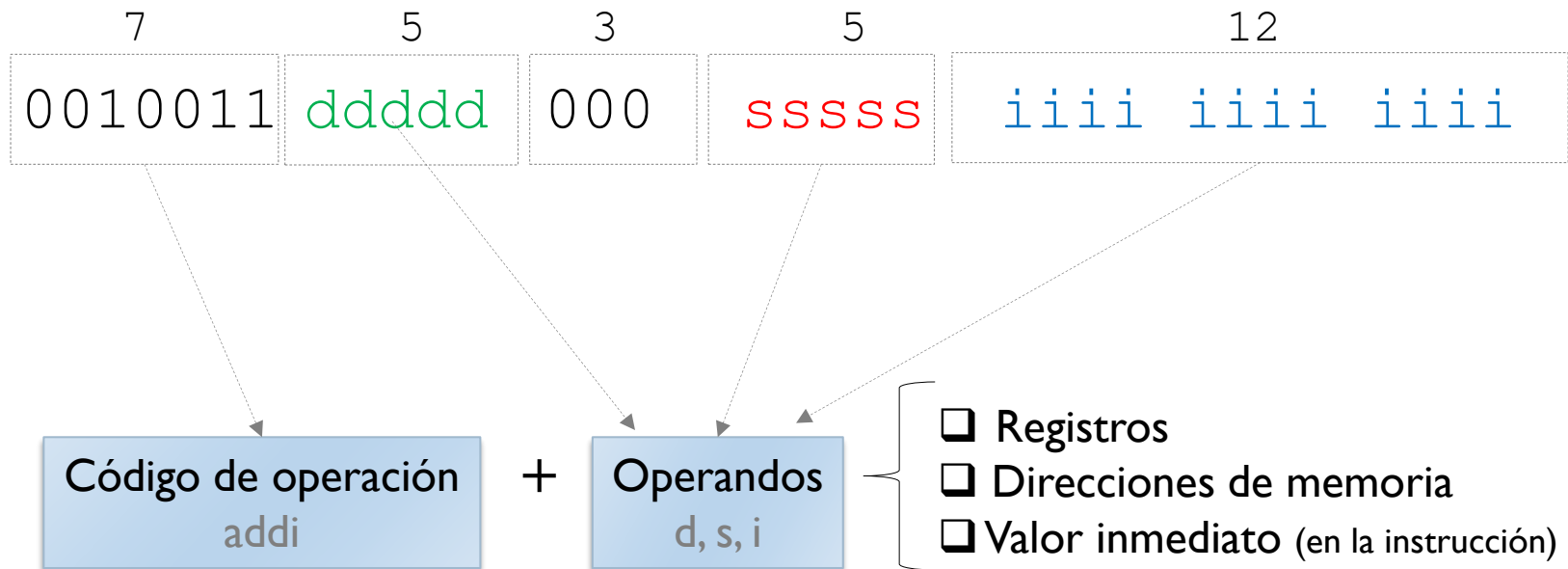
- **Instrucción máquina:** acción elemental que puede ejecutar directamente el procesador

- Ej.: $Rd = Rs + Imm$ # `addi Rd, Rs, Imm`

0010011 dddddd 000 sssss iiiii iiiii iiiii

Instrucciones máquina: propiedades

- ▶ Realizan una **única y sencilla tarea**
- ▶ Operan sobre un **número fijo de operandos**
- ▶ **Incluyen toda la información necesaria para su ejecución**



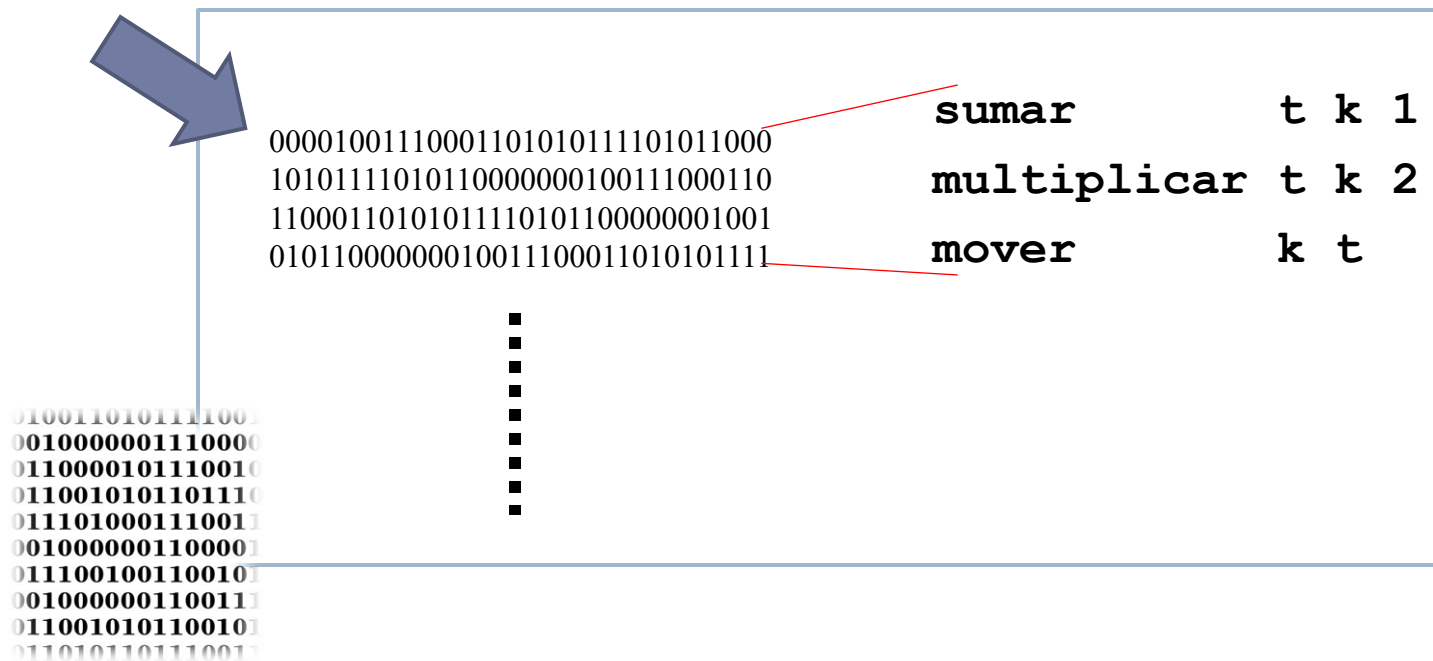
Instrucción máquina: información incluida

- ▶ La **operación a realizar**.
- ▶ Dónde se encuentran los **operandos**:
 - ▶ En registros
 - ▶ En memoria
 - ▶ En la propia instrucción (inmediato)
- ▶ Dónde dejar los resultados (como operando)
- ▶ Una referencia a la siguiente instrucción a ejecutar
 - ▶ De forma implícita, la siguiente instrucción
 - ▶ Un programa es una secuencia consecutiva de instrucciones máquina
 - ▶ De forma explícita en las instrucciones de bifurcación (como operando)



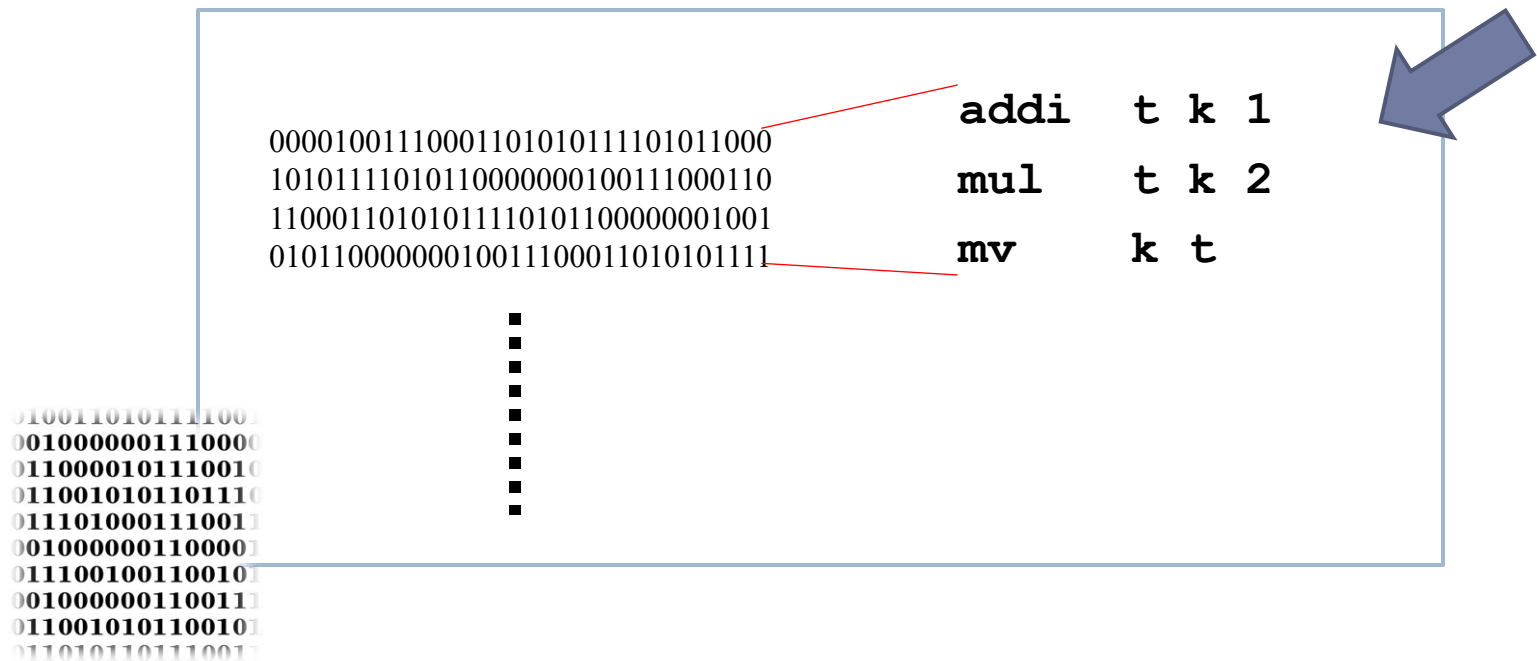
Definición de programa

- **Programa:** lista ordenada de instrucciones máquina que se ejecutan en secuencia (por defecto).



Definición de lenguaje ensamblador

- **Lenguaje ensamblador**: lenguaje legible por un programador que constituye la representación más directa del código máquina específico de una arquitectura de computadoras.



Definición de lenguaje ensamblador

- ▶ **Lenguaje ensamblador:** lenguaje legible por un programador que constituye la representación más directa del código máquina específico de una arquitectura de computadoras.
 - ▶ Emplea códigos nemónicos para representar instrucciones:
 - ▶ `add` – suma
 - ▶ `li` – carga un dato inmediato
 - ▶ Emplea nombres simbólicos para designar a datos y referencias:
 - ▶ `t0` – identificador de un registro
 - ▶ `(t0)` – identificador una posición de memoria
 - ▶ Cada instrucción en ensamblador se corresponde con una instrucción máquina:
 - ▶ `add t1, t2, t3`

Diferentes niveles de lenguajes

Lenguaje de alto nivel
(Ej.: C, C++, etc.)

Compilador

Lenguaje ensamblador
(Ej.: RISC-V)

Ensamblador

Lenguaje máquina
(Ej.: RISC-V)

```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

```
lw    t0, 0(x2)  
lw    t1, 4(x2)  
sw    t1, 0(x2)  
sw    t0, 4(x2)
```

```
0000 1001 1100 0110 1010 1111 0101 1000  
1010 1111 0101 1000 0000 1001 1100 0110  
1100 0110 1010 1111 0101 1000 0000 1001  
0101 1000 0000 1001 1100 0110 1010 1111
```

Definición de juego de instrucciones

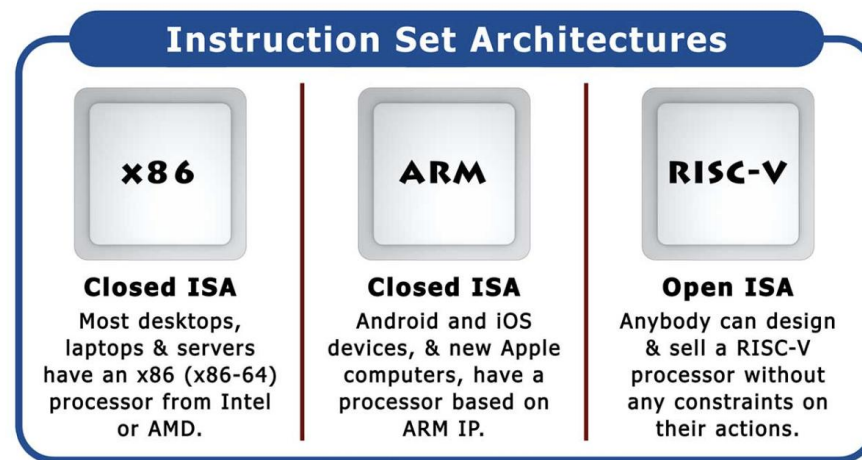
▶ **Instruction Set Architecture (ISA)**

- ▶ Conjunto de instrucciones de un procesador
- ▶ Frontera entre el hardware y el software

▶ Ejemplos:

- ▶ x86, x86-64, ARM, MIPS, RISC-V, PowerPC, Etc.

▶ Tipos:



Características de un juego de instrucciones (1/2)

- ▶ **Formato y codificación del juego de instrucciones**
 - ▶ Instrucciones de longitud fija o variable
 - ▶ 80x86: variable de 1 a 18 bytes
 - ▶ RISC-V, ARM: fijo
- ▶ **Operandos**
 - ▶ Registros, memoria, la propia instrucción
- ▶ **Tipo y tamaño de los operandos**
 - ▶ bytes: 8 bits
 - ▶ enteros: 16, 32, 64 bits
 - ▶ números en coma flotante: simple precisión, doble,...
- ▶ **Modos de direccionamiento**
 - ▶ Especifican el lugar y la forma de acceder a los operandos (registro, memoria o la propia instrucción)

Características de un juego de instrucciones (2/2)

▶ Operaciones

- ▶ Aritméticas, lógicas, de transferencia, control, ...

▶ Instrucciones de control de flujo

- ▶ Saltos incondicionales
- ▶ Saltos condicionales
- ▶ Llamadas a procedimientos

▶ Direccionamiento de la memoria

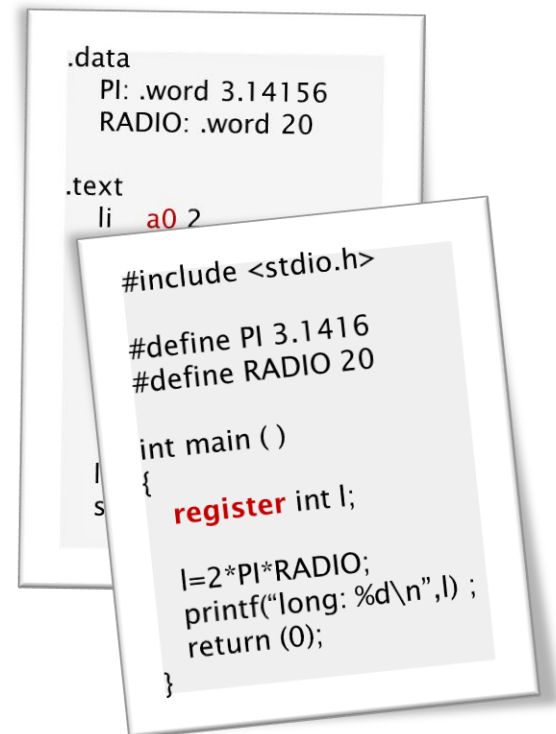
- ▶ La mayoría utilizan direccionamiento por bytes
- ▶ Ofrecen instrucciones para acceder a elementos de varios bytes a partir de una determinada posición

Modelo de programación de un computador

- ▶ Un computador ofrece un modelo de programación formando por:
 - ▶ **Juego de instrucciones (lenguaje ensamblador)**
 - ▶ *ISA: Instruction Set Architecture*
 - ▶ Una instrucción incluye:
 - Código de operación
 - Otros elementos: id. de registros, direcciones de memoria o números
 - ▶ **Elementos de almacenamiento**
 - ▶ Registros
 - ▶ Memoria
 - ▶ Registros de los controladores de E/S
 - ▶ **Modos de ejecución**

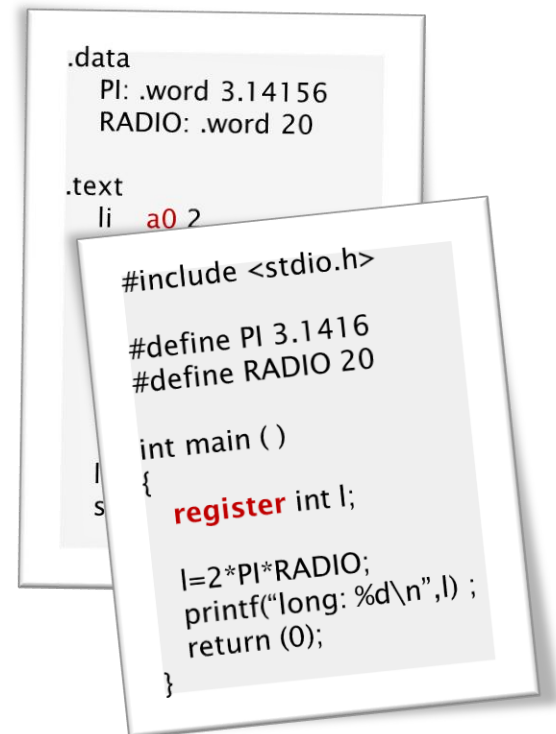
Objetivos

- ▶ Poder escribir y entender pequeños programas en ensamblador
- ▶ Saber cómo se representan los elementos de un lenguaje de alto nivel en ensamblador:
 - ▶ Tipos de datos (int, char, ...)
 - ▶ Estructuras de control (if, while, ...)
- ▶ Trabajar con hardware abierto simulado:
 - ▶ Ensamblador RISC-V
 - ▶ Trabajo con simulador CREATOR

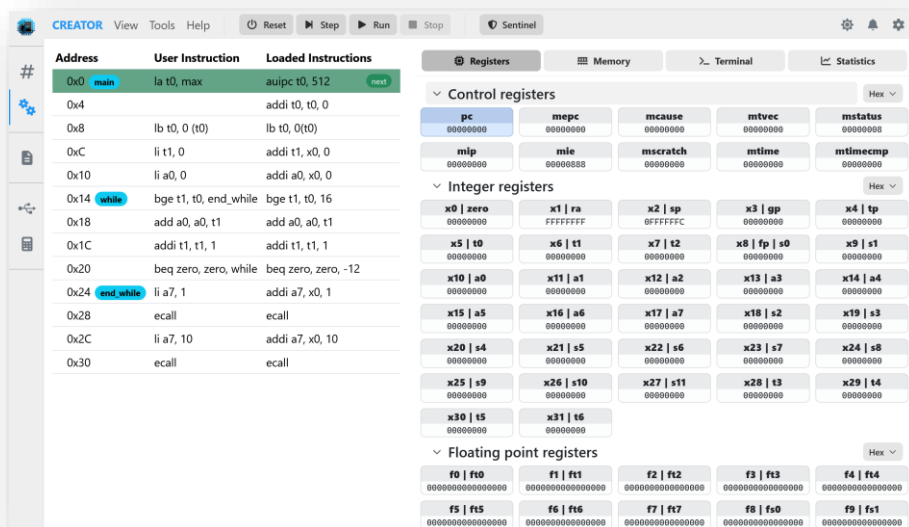


Motivación para aprender ensamblador

- ▶ Comprender qué ocurre cuando un computador ejecuta una sentencia de un lenguaje de alto nivel.
 - ▶ C, C++, Java, Python,...
- ▶ Poder determinar el impacto en tiempo de ejecución de una instrucción de alto nivel.
- ▶ Útil en dominios específicos:
 - ▶ Compiladores
 - ▶ Sistemas Operativos
 - ▶ Juegos
 - ▶ Sistemas empuotrados
 - ▶ Etc.



Motivación para usar CREATOR

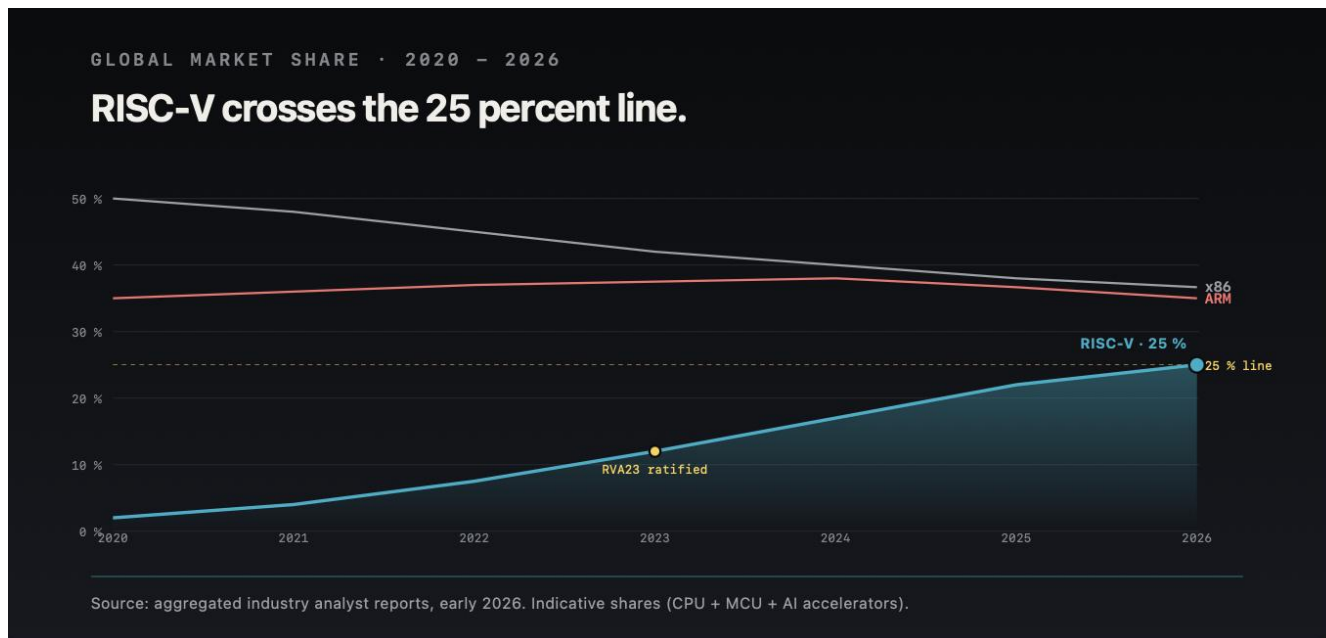


<https://creatorsim.github.io/>

- ▶ CREATOR: *didaCtic*
geneRic assEmbly
progrAmming simulaTOR
- ▶ CREATOR puede simular las arquitecturas RISC-V y MIPS₃₂
- ▶ CREATOR puede ejecutarse desde Firefox, Chrome o Edge

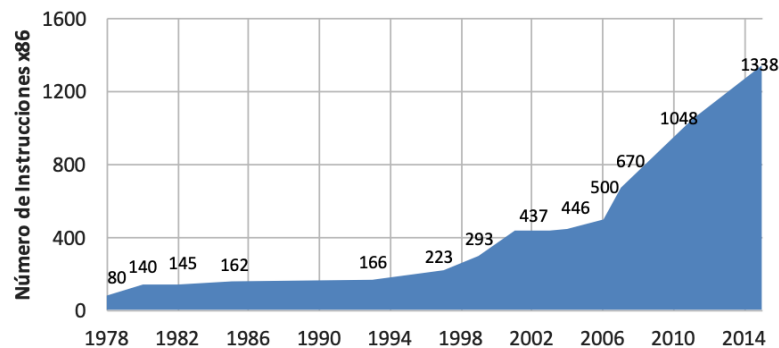
Ventajas de usar RISC-V (1/2)

- ▶ Arquitectura de hardware libre:
 - ▶ Permite que cualquiera trabaje con chips y software de RISC-V
- ▶ Procesador RISC (*Reduced Instruction Set Computer*)
 - ▶ Otros: ARM, MIPS, etc.
- ▶ Rápida adopción:



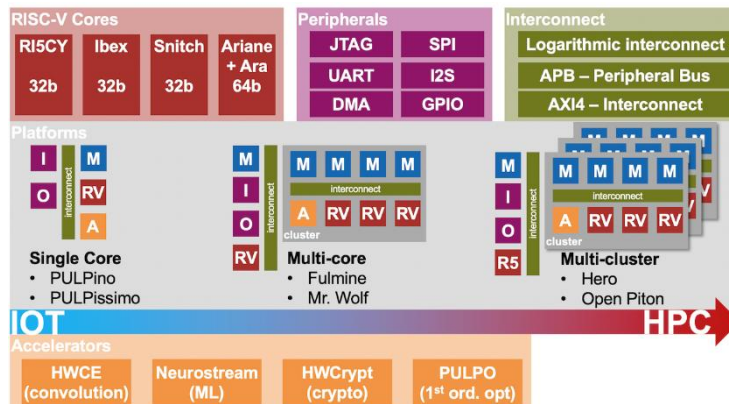
Ventajas de usar RISC-V (2/2)

- ▶ Conjunto de instrucciones base pequeño y sencillo
 - ▶ RV32I -> ~47 instrucciones, RV32IMAF -> ~76
 - ▶ Diferencia con instrucciones de la arquitectura x86



Guía Práctica de RISC-V.
David Patterson y Andrew Waterman

- ▶ Conjuntos de instrucciones modular:



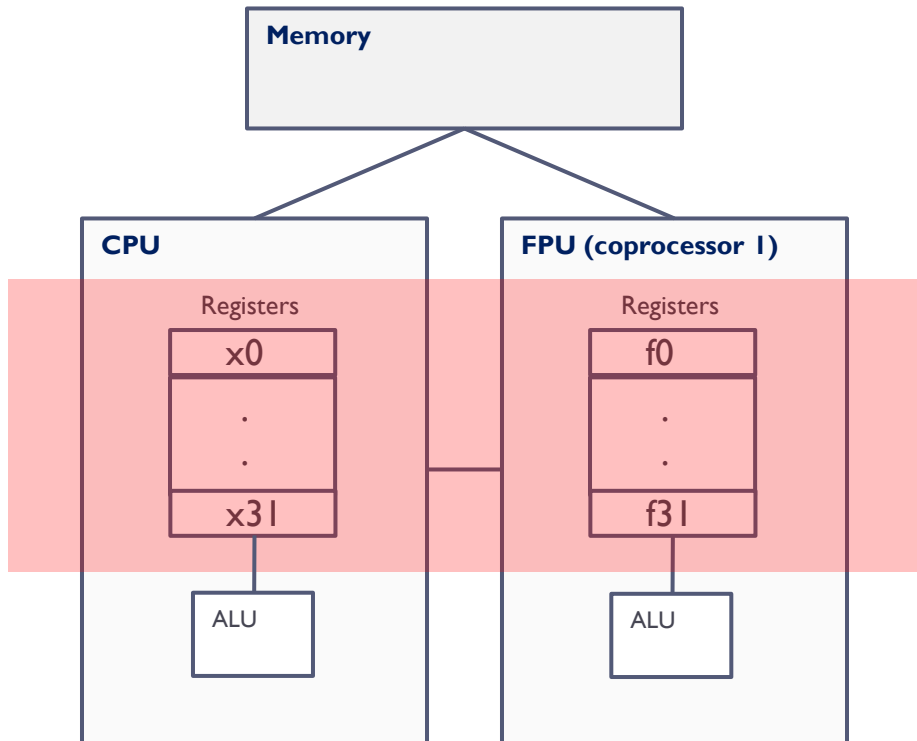
Juegos de instrucciones RISC-V

- ▶ Juegos de instrucciones:
 - ▶ RV32I: Juego de instrucciones sobre enteros. 32 bits
 - ▶ RV64I: Juego de instrucciones sobre enteros. 64 bits
 - ▶ RV128I: Juego de instrucciones sobre enteros. 128 bits
- ▶ Sobre cada uno de ellos hay diferentes extensiones:
 - ▶ M: instrucciones para multiplicación y división de enteros
 - ▶ F: instrucciones para coma flotante de simple precisión
 - ▶ D: instrucciones para coma flotante de doble precisión
 - ▶ G: Incluye M, F y D
 - ▶ Q: instrucciones de coma flotante de cuádruple precisión
 - ▶ Etc.
- ▶ Ejemplo: RV64IF: procesador RISC-V de 64 bits con instrucciones de coma flotante de simple precisión

Contenidos

1. Fundamentos básicos de la programación en ensamblador
 1. Motivación y objetivos
 2. Introducción al ensamblador RISC-V 32
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Aritméticas y lógicas
 - ▶ Desplazamientos
 - ▶ Bifurcaciones o saltos
2. Ensamblador RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
5. Llamadas a procedimientos y uso de la pila

RISC-V 32



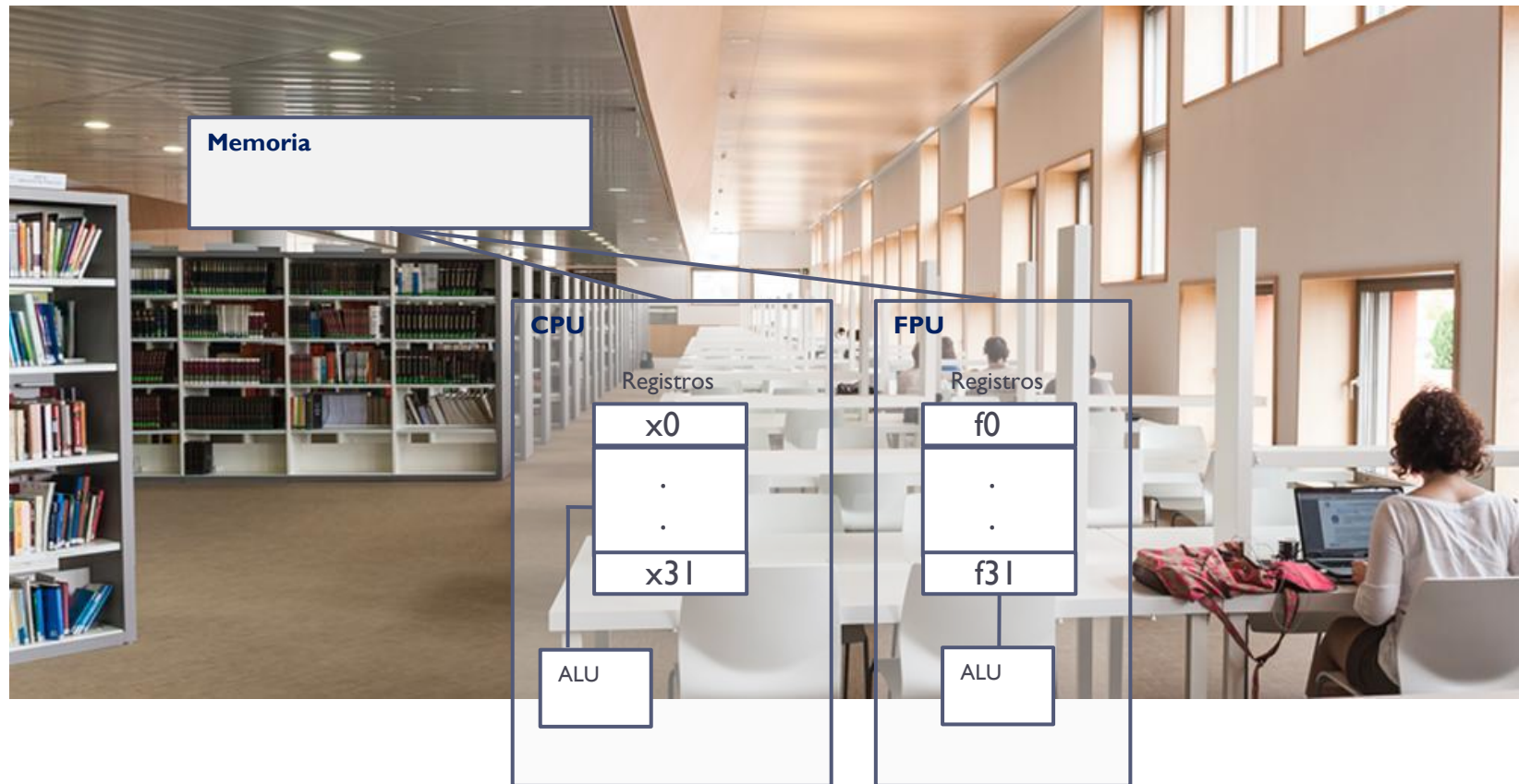
▶ Registros

- ▶ Dentro del procesador
- ▶ Instrucciones trabajan con valores en registros
- ▶ 32 registros de 32 bits

▶ Memoria

- ▶ Fuera del procesador
- ▶ Intercambio de datos a/desde registros
- ▶ Más capacidad pero más tiempo de acceso que los registros
- ▶ Direcciones de 32 bits
 - ▶ 4 GiB direccionable

RISC-V: memoria vs banco de registros



Banco de registros del RISC-V 32

0		■ Hay 32 registros		16
1		□ 4 bytes de tamaño (una palabra)		17
2		□ Se nombran con un x al principio		18
3				19
4		■ Convenio de uso		20
5		□ Reservados		21
6		□ Argumentos		22
7		□ Resultados		23
8		□ Temporales		24
9		□ Punteros		25
10				26
11				27
12				28
13				29
14				30
15				31

Banco de registros del RISC-V 32

0		zero		16
1				17
2				18
3				19
4				20
5				21
6				22
7				23
8				24
9				25
10				26
11				27
12				28
13				29
14				30
15				31

Valor cableado a cero
No puede modificarse

Banco de registros del RISC-V 32

0		zero			16
1					17
2					18
3					19
4					20
5		t0			21
6		t1			22
7		t2			23
8					24
9					25
10					26
11					27
12			t3		28
13			t4		29
14			t5		30
15			t6		31

Valores temporales

Banco de registros del RISC-V 32

0		zero		16
1				17
2				18
3				19
4				20
5		t0	s2	21
6		t1	s3	22
7		t2	s4	23
8		s0 / fp	s5	24
9		s1	s6	25
10			s7	26
11			s8	27
12			s9	28
13			s10	29
14			s11	30
15			t3	31
			t4	
			t5	
			t6	

Valores guardados

Banco de registros del RISC-V 32

0		zero			
1		ra	a6		16
2			a7		17
3			s2		18
4			s3		19
5		t0	s4		20
6		t1	s5		21
7		t2	s6		22
8		s0 / fp	s7		23
9		s1	s8		24
10		a0	s9		25
11		a1	s10		26
12		a2	s11		27
13		a3	t3		28
14		a4	t4		29
15		a5	t5		30
			t6		31

Paso de parámetros y
Gestión de subrutinas

Banco de registros del RISC-V 32

0		zero	a6		16
1		ra	a7		17
2		sp	s2		18
3		gp	s3		19
4		tp	s4		20
5		t0	s5		21
6		t1	s6		22
7		t2	s7		23
8		s0 / fp	s8		24
9		s1	s9		25
10		a0	s10		26
11		a1	s11		27
12		a2	t3		28
13		a3	t4		29
14		a4	t5		30
15		a5	t6		31

Puntos de lectura/escritura (punteros)

Banco de registros (enteros)

resumen

Nombre ABI	Número	Uso
zero	x0	Constante 0
ra	x1	Dirección de retorno (rutinas)
sp	x2	Puntero a pila
gp	x3	Puntero al área global
tp	x4	Puntero al hilo
t0...t2	x5-x7	Temporal (<u>NO</u> se conserva entre llamadas)
s0 / fp	x8	Temporal (se conserva entre llamadas) / Puntero a marco de pila
s1	x9	Temporal (se conserva entre llamadas)
a0...a1	x10...11	Argumento de entrada para rutinas/valores de retorno
a2...a7	12...x17	Argumento de entrada para rutinas
s2... s11	x18...x27	Temporal (se conserva entre llamadas)
t3...t6	x28...x31	Temporal (<u>NO</u> se conserva entre llamadas)

- ▶ Hay 32 registros
 - ▶ 4 bytes de tamaño (una palabra)
 - ▶ Doble nombrado:
 - ▶ Lógico: nombre ABI (*Application Binary Interface*)
 - ▶ Numérico: con **x** al principio
- ▶ Convenio de uso
 - ▶ Reservados
 - ▶ Argumentos
 - ▶ Resultados
 - ▶ Temporales
 - ▶ Punteros

Banco de registros (coma flotante)

resumen

Nombre registro	Número registro	Uso
ft0-ft7	f0 ... f7	Temporales (como los t...)
fs0-fs1	f8 ... f9	Se guardan (como los s...)
fa0-fa1	f10 ... f11	Argumentos/retorno (como los a...)
fa2-fa7	f12 ... f17	Argumentos (como los a...)
fs2-fs11	f18 ... f27	Se guardan (como los s...)
ft8-ft11	f28 ... f31	Temporales (como los t...)

- ▶ Hay 32 registros
- ▶ En R32F (simple precisión) los registros son de 32 bits (4 bytes)
- ▶ En R32D (doble precisión) los registros son de 64 bits (8 bytes) y pueden almacenar:
 - ▶ Valores de simple precisión en los 32 bits inferiores del registro
 - ▶ Valores de doble precisión en todos los 64 bits del registro

CREATOR

Banco de registros

The screenshot displays the CREATOR simulator interface. On the left, a table lists instructions at various memory addresses. The 'Registers' panel on the right shows the current state of the processor's registers. A callout box labeled 'Banco de registros' points to this panel.

Address	User Instruction	Loaded Instructions
0x0	li t0, 10	addi t0, x0, 10
0x4	li t1, 13	addi t1, x0, 13
0x8	li t2, 45	addi t2, x0, 45
0xC	li t3, 33	addi t3, x0, 33
0x10	add t4, t0, t1	add t4, t0, t1
0x14	sub t4, t2, t3	sub t4, t2, t3
0x18	mul t4, t3, t3	mul t4, t3, t3
0x1C	div t4, t2, t0	div t4, t2, t0
0x20	mv a0, t4	addi a0, t4, 0
0x24	li a7, 1	addi a7, x0, 1
0x28	ecall	ecall
0x2C	li a7, 10	addi a7, x0, 10
0x30	ecall	ecall

Registers Panel:

- Control registers:** x0 | zero (0), x1 | ra (-1), x2 | sp (268435452), x3 | gp (0), x4 | tp (0).
- Integer registers:**
 - x5 | t0 (10), x6 | t1 (13), x7 | t2 (45), x8 | fp | s0 (0), x9 | s1 (0).
 - x10 | a0 (0), x11 | a1 (0), x12 | a2 (0), x13 | a3 (0), x14 | a4 (0).
 - x15 | a5 (0), x16 | a6 (0), x17 | a7 (0), x18 | s2 (0), x19 | s3 (0).
 - x20 | s4 (0), x21 | s5 (0), x22 | s6 (0), x23 | s7 (0), x24 | s8 (0).
 - x25 | s9 (0), x26 | s10 (0), x27 | s11 (0), x28 | t3 (33), x29 | t4 (0).
 - x30 | t5 (0), x31 | t6 (0).
- Floating point registers:** (Collapsed)

<https://creatorsim.github.io/>

Contenidos

1. Fundamentos básicos de la programación en ensamblador
 1. Motivación y objetivos
 2. Introducción al ensamblador RISC-V 32
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Aritméticas y lógicas
 - ▶ Desplazamientos
 - ▶ Bifurcaciones o saltos
2. Ensamblador RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
5. Llamadas a procedimientos y uso de la pila

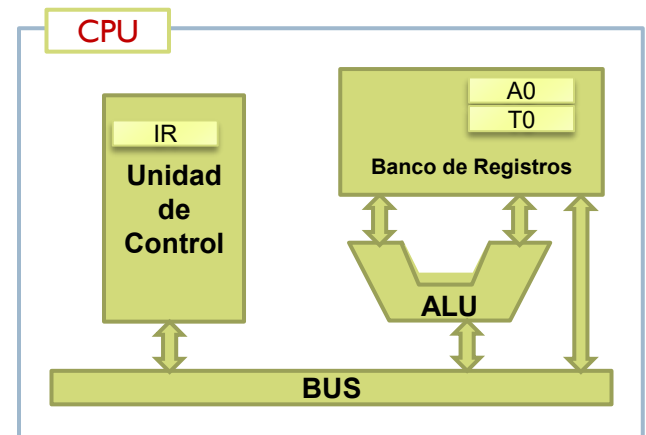
Transferencia de datos (registros enteros)

► Copia **datos**:

- entre **registros**
- entre **registros y memoria**

► Ejemplos:

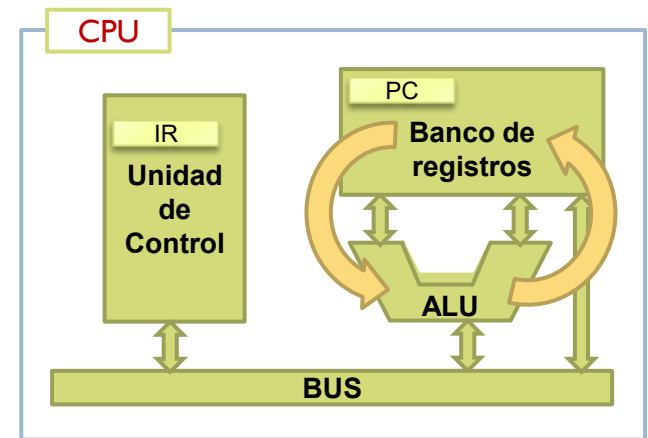
- Copia de registro a registro
mv a0, t0
- Carga inmediata
li t0, 5



```
mv    a0, t0    # a0 ← t0
li    t0, 5     # t0 ← 000....00101
```

Aritméticas (registros enteros)

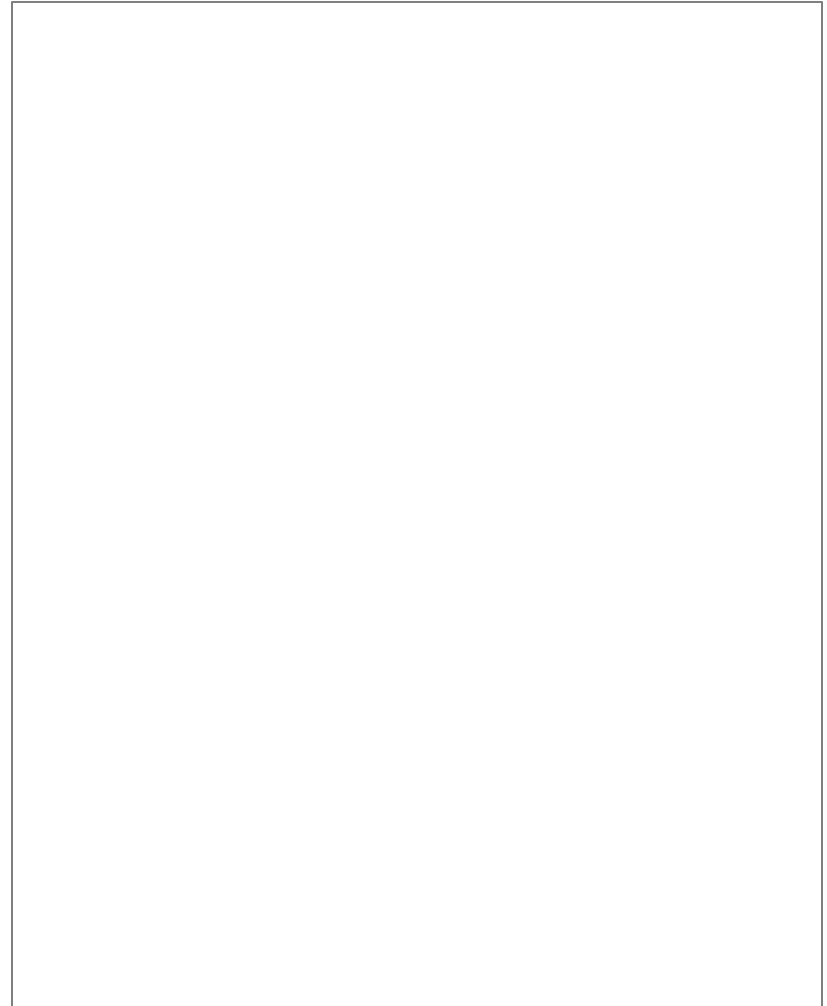
- ▶ Realiza operaciones aritméticas de enteros en Complementos a dos
- ▶ Ejemplos (ALU):
 - ▶ Sumar
`add t0, t1, t2` $\# t0 \leftarrow t1 + t2$
`addi t0, t1, 5` $\# t0 \leftarrow t1 + 5$
 - ▶ Restar
`sub t0, t1, t2` $\# t0 \leftarrow t1 - t2$
 - ▶ Multiplicar
`mul t0, t1, t2` $\# t0 \leftarrow t1 * t2$
 - ▶ División entera (5 / 2=2)
`div t0, t1, t2` $\# t0 \leftarrow t1 / t2$
 - ▶ Resto de la división (5 % 2=1)
`rem t0, t1, t2` $\# t0 \leftarrow t1 \% t2$



Ejemplo

```
int a = 5;  
int b = 7;  
int c = 8;  
int i;
```

```
i = a * (b + c)
```



Ejemplo

```
int a = 5;  
int b = 7;  
int c = 8;  
int i;
```

```
i = a * (b + c)
```

```
li    t1, 5 # t1:a  
li    t2, 7 # t2:b  
li    t3, 8 # t3:c
```

```
add   t4, t2, t3  
mul   t4, t4, t1
```

Ejercicio

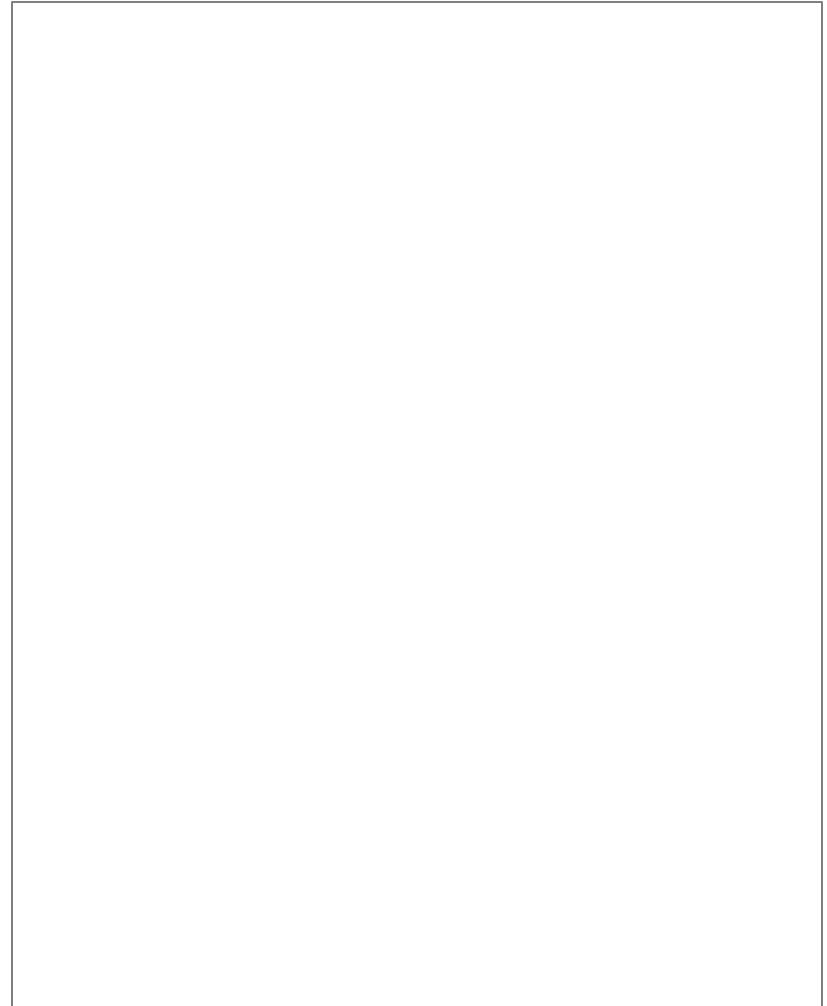
```
int a = 5;
```

```
int b = 7;
```

```
int c = 8;
```

```
int i;
```

```
i = -(a * (b - 10) + c)
```



Ejercicio (solución)

```
int a = 5;  
int b = 7;  
int c = 8;  
int i;
```

```
i = -(a * (b - 10) + c)
```

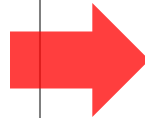
```
li    t1, 5 # t1:a  
li    t2, 7 # t2:b  
li    t3, 8 # t3:c
```

```
li    t0, 10  
sub    t4, t2, t0  
mul    t4, t4, t1  
add    t4, t4, t3  
li    t0, -1  
mul    t4, t4, t0
```

Ejercicio

```
li  t1, 5  
li  t2, 7  
li  t3, 8
```

```
li  t0, 10  
sub t4, t2, t0  
mul t4, t4, t1  
add t4, t4, t3  
li  t0, -1  
mul t4, t4, t0
```

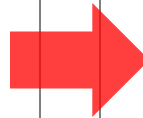


**¿Y usando menos
instrucciones?**

Ejercicio (solución)

```
li t1, 5  
li t2, 7  
li t3, 8
```

```
li t0, 10  
sub t4, t2, t0  
mul t4, t4, t1  
add t4, t4, t3  
li t0, -1  
mul t4, t4, t0
```



```
li t1, 5  
li t2, 7  
li t3, 8
```

```
addi t4, t2, -10  
  
mul t4, t4, t1  
add t4, t4, t3  
sub t4, x0, t4
```

Lógicas

- ▶ Operaciones **booleanas**

- ▶ Ejemplos:

- ▶ AND

`and t0, t1, t2` ($t0 = t1 \& t2$)
`andi t0, t1, t2` ($t0 = t1 \& t2$)

- ▶ OR

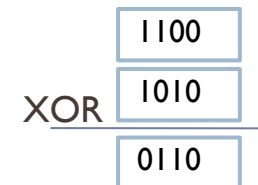
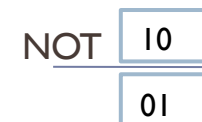
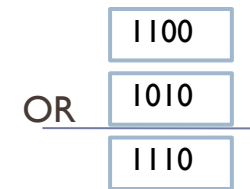
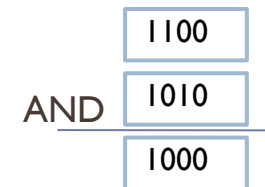
`or t0, t1, t2` ($t0 = t1 | t2$)
`ori t0, t1, 80` ($t0 = t1 | 80$)

- ▶ NOT

`not t0, t1` ($t0 = ! t1$)
`xori t0 t1 -1`

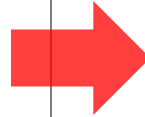
- ▶ XOR

`xor t0, t1, t2` ($t0 = t1 \wedge t2$)



Ejercicio

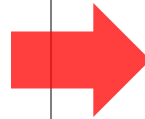
```
li t0, 5  
li t1, 0x007FFFFF  
  
and t2, t1, t0
```



¿Cuál será el valor
almacenado en t2?

Ejercicio (solución)

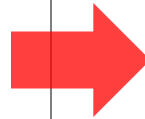
```
li t0, 5  
li t1, 0x007FFFFFFF  
  
and t2, t1, t0
```



	00...0101	t0
	00...1111	t1
and	00...0101	t2

Ejercicio

```
li t0, 5  
li t1, 0x007FFFFFFF  
  
and t2, t1, t0
```

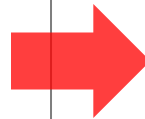


¿Qué permite hacer un and
con 0x007FFFFFFF?

Ejercicio (solución)

```
li t0, 5
li t1, 0x007FFFFFFF

and t2, t1, t0
```



¿Qué permite hacer un and con 0x007FFFFFFF?

Obtener los 23 bits menos significativos

La constante usada para la selección de bits se denomina **máscara**.

Contenidos

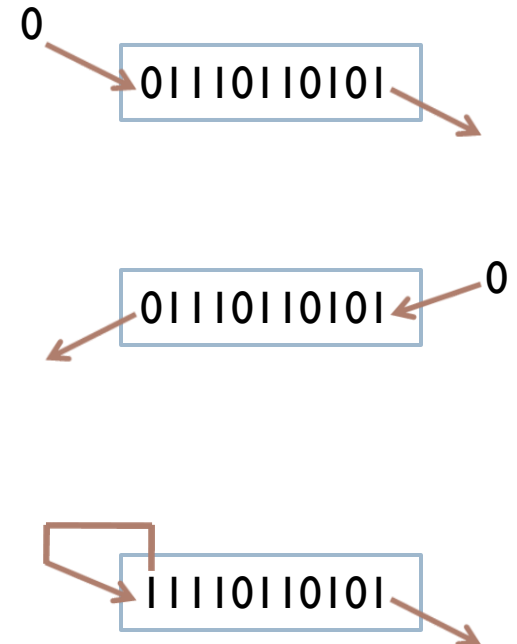
1. Fundamentos básicos de la programación en ensamblador
 1. Motivación y objetivos
 2. Introducción al ensamblador RISC-V 32
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Aritméticas y lógicas
 - ▶ Desplazamientos
 - ▶ Bifurcaciones o saltos
2. Ensamblador RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
5. Llamadas a procedimientos y uso de la pila

Desplazamientos

- ▶ De movimiento de **bits**.
Solo sobre registros enteros.

- ▶ Ejemplos:

- ▶ Desplazamiento **lógico** a la **derecha**
`srli t0, t0, 4` ($t0 = t0 \gg 4$ bits)
- ▶ Desplazamiento **lógico** a la **izquierda**
`slli t0, t0, 5` ($t0 = t0 \ll 5$ bits)
- ▶ Desplazamiento **aritmético**
`srai t0, t0, 2` ($t0 = t0 \gg 2$ bits)

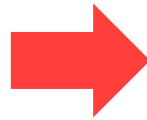


Ejercicio (solución)

```
li t0, 5
```

```
li t1, 6
```

```
srai t0, t1, 1
```



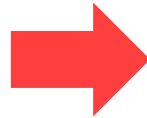
- ¿Cuál es el valor de t0?

000 0110 t1

Se desplaza 1 bit a la derecha (/2)

000 0011 t0

```
slli t0, t1, 1
```



- ¿Cuál es el valor de t0?

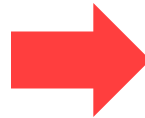
000 0110 t1

Se desplaza 1 bit a la izquierda (x2)

000 1100 t0

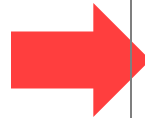
Ejercicio

Realice un programa que detecte el signo de un número almacenado t0 y deje en t1 un 1 si es negativo y un 0 si es positivo



Ejercicio (solución)

Realice un programa que detecte el signo de un número almacenado t0 y deje en t1 un 1 si es negativo y un 0 si es positivo



```
li    t0, -3  
srli  t1, t0, 31
```

Contenidos

1. Fundamentos básicos de la programación en ensamblador
 1. Motivación y objetivos
 2. Introducción al ensamblador RISC-V 32
 - ▶ Registros
 - ▶ Transferencia de datos
 - ▶ Aritméticas y lógicas
 - ▶ Desplazamientos
 - ▶ Bifurcaciones o saltos
2. Ensamblador RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
5. Llamadas a procedimientos y uso de la pila

Instrucciones de comparación (registros enteros)

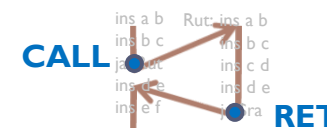
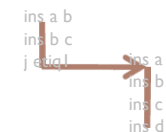
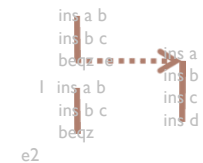
▶ slt rd, rs1, rs2	if (s(rs1) < s(rs2))	rd = 1; else rd = 0
▶ sltu rd, rs1, rs2	if (u(rs1) < u(rs2))	rd = 1; else rd = 0
▶ slti rd, rs1, 5	if (s(rs1) < s(5))	rd = 1; else rd = 0
▶ sltiu rd, rs1, 5	if (u(rs1) < u(5))	rd = 1; else rd = 0
▶ seqz rd, rs1	if (rs1 == 0)	rd = 1; else rd = 0
▶ snez rd, rs1	if (rs1 != 0)	rd = 1; else rd = 0
▶ sgtz rd, rs1	if (rs1 > 0)	rd = 1; else rd = 0
▶ sltz rd, rs1	if (rs1 < 0)	rd = 1; else rd = 0

como pseudoinstrucciones

lt: (<) less than
le: (<=) less equal than
gt: (>) greater than
ge: (>=) greater equal than
eqz: (==0) equal zero
nez: (!=0) not equal zero

Control de Flujo

- ▶ Cambio de la secuencia de instrucciones a ejecutar (instrucciones de bifurcación)
- ▶ Distintos tipos:
 - ▶ Bifurcación o salto condicional:
 - ▶ Saltar a la posición etiqueta , si $t0 \neq t1$
 - ▶ Ej.: `bne t0, t1, etiqueta`
 - ▶ Bifurcación o salto incondicional:
 - ▶ El salto se realiza siempre
 - ▶ Ej.: `j etiqueta`
 - ▶ Llamada a procedimiento:
 - ▶ Ej.: `jal ra, subrutina ..... jr ra`



Instrucciones de bifurcación

► Condicional (solo con registros enteros):

- **beq** **t0, t1, etiq** # salta a etiq1 si **t0 == t1**
- **bne** **t0, t1, etiq** # salta a etiq1 si **t0 != t1**
- **blt** **t0, t1, etiq** # salta a etiq1 si **t0 < t1**
- **bltu** **t0, t1, etiq** # salta a etiq1 si **t0 < t1 (unsigned)**
- **bge** **t0, t1, etiq** # salta a etiq1 si **t0 >= t1**
- **bgeu** **t0, t1, etiq** # salta a etiq1 si **t0 >= t1 (unsigned)**
- **bgt** **t0, t1, etiq** # salta a etiq1 si **t0 > t1**
- **ble** **t0, t1, etiq** # salta a etiq1 si **t0 <= t1**

como pseudoinstrucciones

Instrucciones de bifurcación

etiqueta de salto

► Condicional (solo con registros enteros):

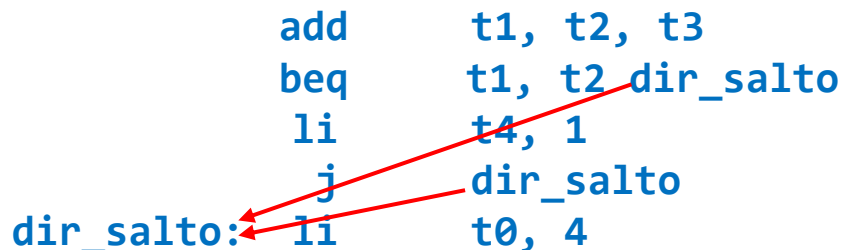
- `beq t0, t1, etiq` # salta a etiq1 si `t0 == t1`
- `bne t0, t1, etiq` # salta a etiq1 si `t0 != t1`
- `blt t0, t1, etiq` # salta a etiq1 si `t0 < t1`
- `bltu t0, t1, etiq` # salta a etiq1 si `t0 < t1` (unsigned)
- `bge t0, t1, etiq` # salta a etiq1 si `t0 >= t1`
- `bgeu t0, t1, etiq` # salta a etiq1 si `t0 >= t1` (unsigned)
- `bgt t0, t1, etiq` # salta a etiq1 si `t0 > t1`
- `ble t0, t1, etiq` # salta a etiq1 si `t0 <= t1`

► Incondicional:

- `j etiq` # salta a etiq. Equivalente a `beq x0 x0 etiq`

`etiq` hace referencia una instrucción (representa a una dirección de memoria donde se encuentra la instrucción) a la que se salta:

```
add    t1, t2, t3
beq     t1, t2, dir_salto
li      t4, 1
j       dir_salto
dir_salto: li t0, 4
```



Estructuras de control

if

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int a=1;
int b=2;

main ()
{
    if (a < b) {
        a = b;
    }
    ...
}
```

Estructuras de control if

CREATOR

WebSIM

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int a=1;
```

```
int b=2;
```

```
main ()
```

```
{  
  if (a < b) {
```

```
    a = b;
```

```
  }
```

```
  ...
```

```
}
```

```
li t1, 1
```

```
li t2, 2
```

```
if_1: blt t1, t2, then_1
```

```
j fin_1
```

```
then_1: mv t1, t2
```

```
fin_1: ...
```

Estructuras de control if

CREATOR

WebSIM

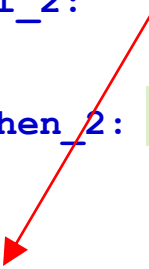
beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int a=1;
int b=2;

main ()
{
    if (a < b) {
        a = b;
    }
    ...
}
```

```
li t1, 1
li t2, 2

if_2: bge t1, t2, fin_2
then_2: mv t1, t2
fin_2: ...
```



Estructuras de control if-else

CREATOR

WebSIM

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int a=1;
int b=2;

main ()
{
    if (a < b){
        // acción 1
    } else {
        // acción 2
    }
}
```

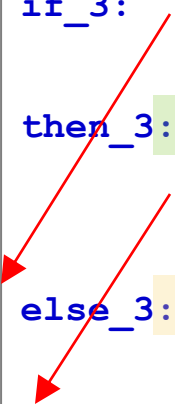
```
li t1, 1
li t2, 2

if_3: bge t1, t2, else_3

then_3: # acción 1
j fi_3

else_3: # acción 2

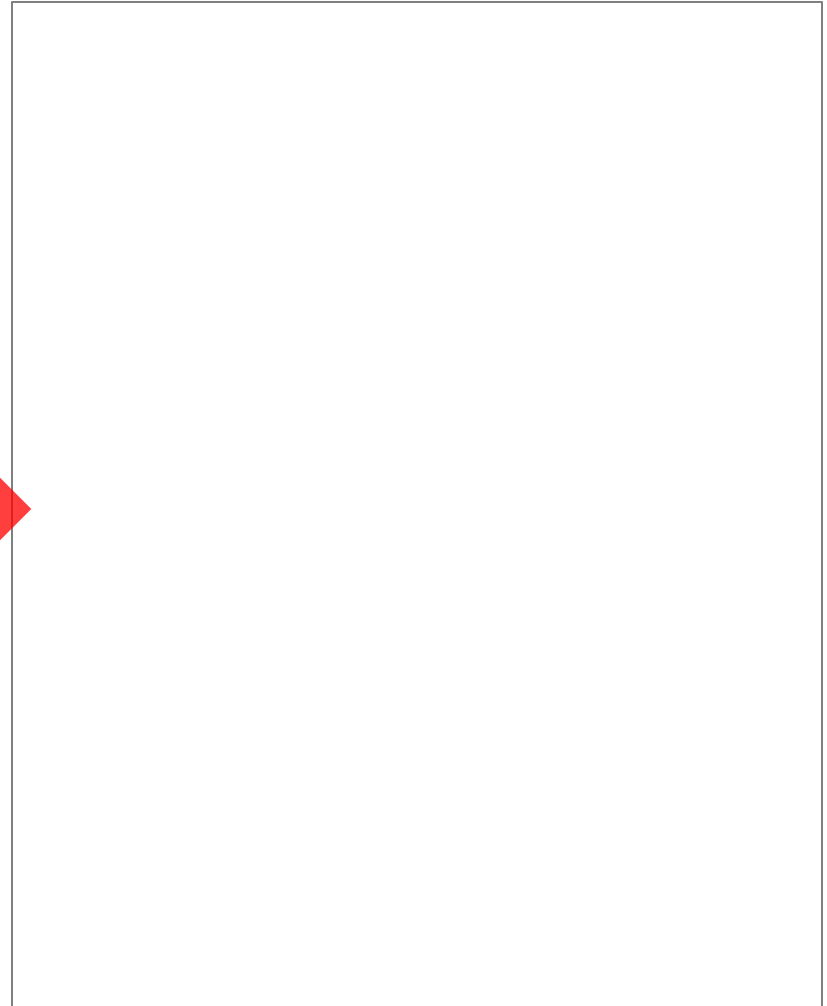
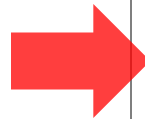
fi_3: ...
```



Ejercicio

```
int b1 = 4;  
int b2 = 2;
```

```
if (b2 == 8) {  
    b1 = 1;  
}  
...
```



Ejercicio (solución)

```
int b1 = 4;  
int b2 = 2;
```

```
if (b2 == 8) {  
    b1 = 1;  
}  
...
```



```
li    t0, 4  
li    t1, 2  
li    t2, 8  
  
bne   t0, t2, fin1  
li    t1, 1  
fin1: ...
```

Estructuras de control

while

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int i;

main ()
{
    i=0;
    while (i < 10) {

        /* acción */
        i = i + 1 ;
    }
}
```

Estructuras de control while

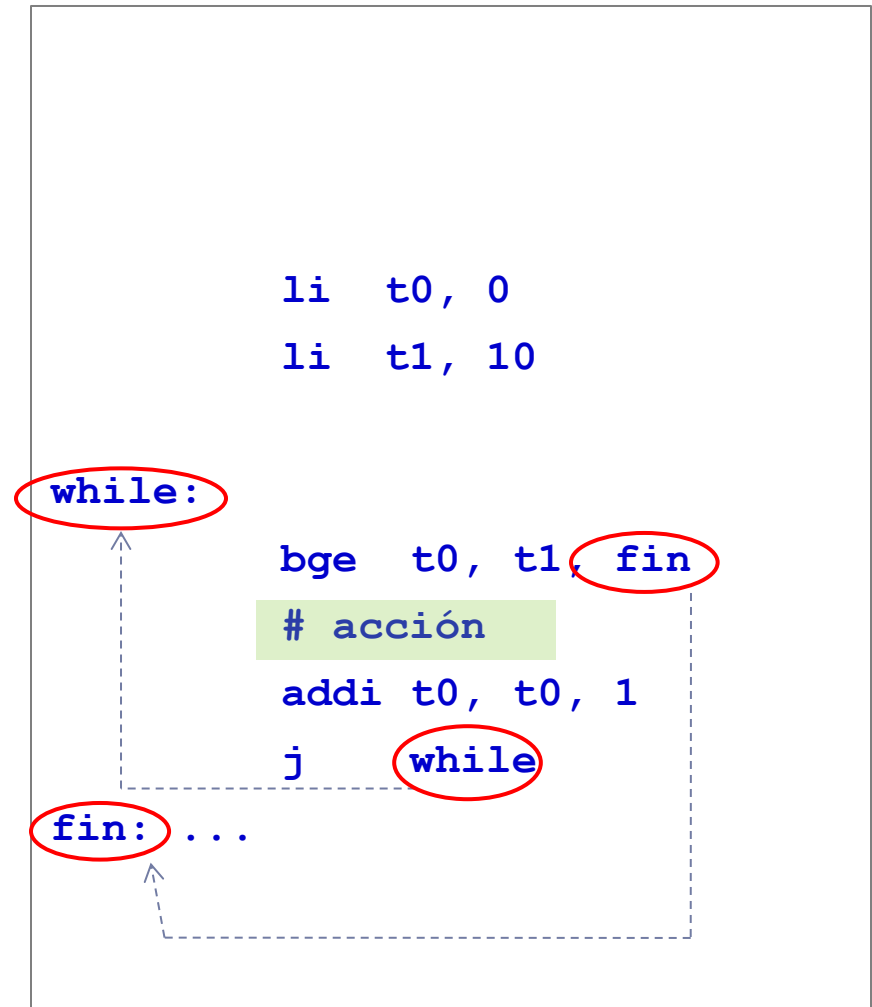
CREATOR

beq	t1 = t0
bne	t1 != t0
bge	t1 >= t0
ble	t0 <= t1
blt	t1 < t0
bgt	t0 > t1

```
int i;

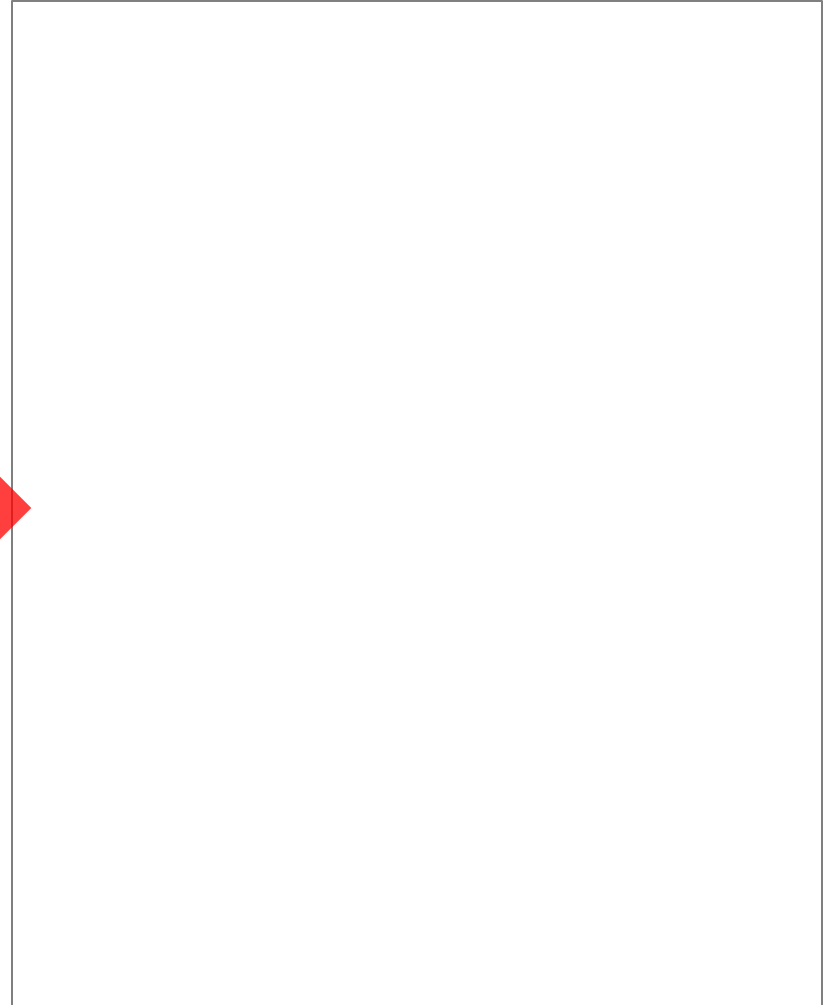
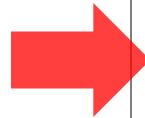
main ()
{
    i=0;
    while (i < 10) {

        /* acción */
        i = i + 1 ;
    }
}
```



Ejercicio

Realice un programa que calcule la suma de los diez primeros números y deje este valor en el registro a0



Ejercicio (solución)

Realice un programa que calcule la suma de los diez primeros números y deje este valor en el registro a0

$1 + 2 + 3 + \dots + 10$



```
li  a0, 0
add a0, a0, 1
add a0, a0, 2
add a0, a0, 3
add a0, a0, 4
add a0, a0, 5
add a0, a0, 6
add a0, a0, 7
add a0, a0, 8
add a0, a0, 9
```

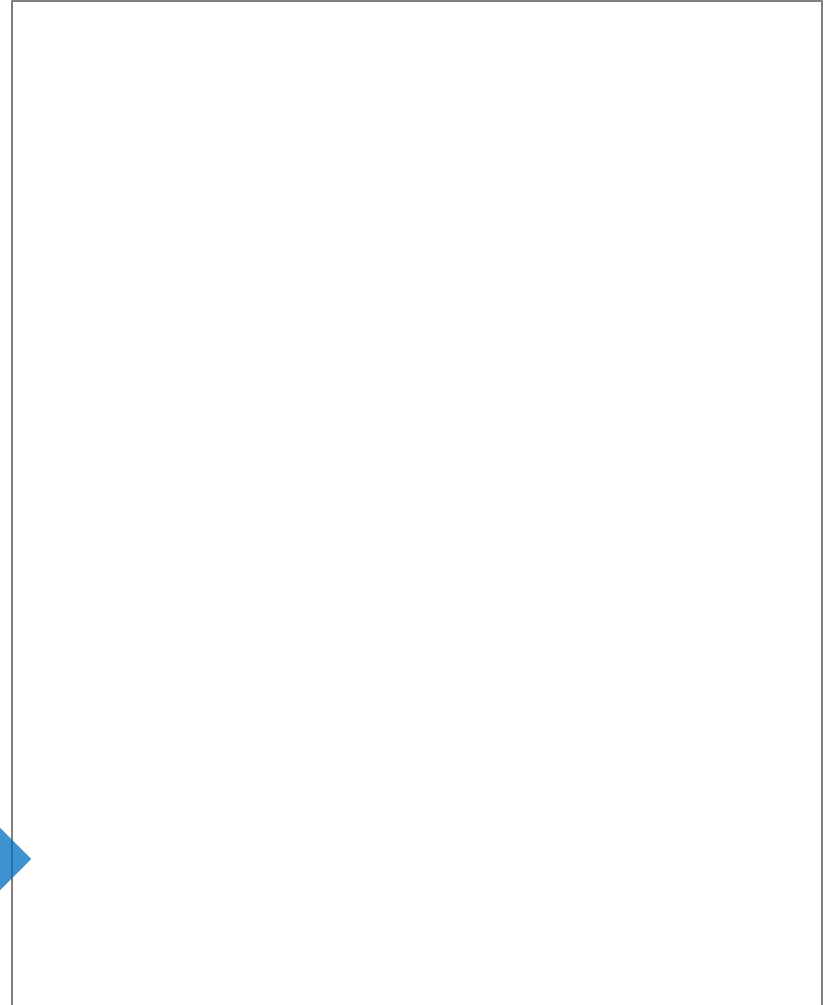
**NO vale
;-)**

Ejercicio (solución)

Realice un programa que calcule la suma de los diez primeros números y deje este valor en el registro a0



```
int i, s;  
  
s=0;  
i=0;  
while (i <= 10)  
{  
    s = s + i ;  
    i = i + 1 ;  
}
```

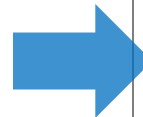


Ejercicio (solución)

Realice un programa que calcule la suma de los diez primeros números y deje este valor en el registro a0



```
int i, s;  
  
s=0;  
i=0;  
while (i <= 10)  
{  
    s = s + i ;  
    i = i + 1 ;  
}
```



```
li t0, 0  
li a0, 0  
li t2, 10  
  
while1:  
    bgt t0, t2, fin1  
    add a0, a0, t0  
    addi t0, t0, 1  
    j while1  
  
fin1:
```

Ejercicio

- ▶ Calcular el número de 1's que hay en un registro (t0).
Resultado en t3

Ejercicio (solución)

- Calcular el número de 1's que hay en un registro (t0).
Resultado en t3

```
i = 0;
n = 45;  # número
s = 0;
while (i < 32)
{
    b = primer bit de n
    s = s + b;
    desplazar el contenido
    de n un bit a la
    derecha
    i = i + 1 ;
}
```

Ejercicio (solución)

- Calcular el número de 1's que hay en un registro (t0).
Resultado en t3

C/C++

```
i = 0;
n = 45;  # número
s = 0;
while (i < 32)
{
    b = primer bit de n
    s = s + b;
    desplazar el contenido
        de n un bit a la
        derecha
    i = i + 1 ;
}
```

```
i = 0;
n = 45;  // número
s = 0;
while (i < 32)
{
    b = n & 1;
    s = s + b;
    n = n >> 1;
    i = i + 1 ;
}
```

Ejercicio (solución)

- Calcular el número de 1's que hay en un registro (t0).
Resultado en t3

Python

```
i = 0;
n = 45;  # número
s = 0;
while (i < 32)
{
    b = primer bit de n
    s = s + b;
    desplazar el contenido
        de n un bit a la
        derecha
    i = i + 1 ;
}
```

```
i = 0
n = 45
s = 0
while (i < 32):
    b = n & 1
    s = s + b
    n = n >> 1
    i = i + 1

print(s)
```

Ejercicio (solución)

- Calcular el número de 1's que hay en un registro (t0).
Resultado en t3

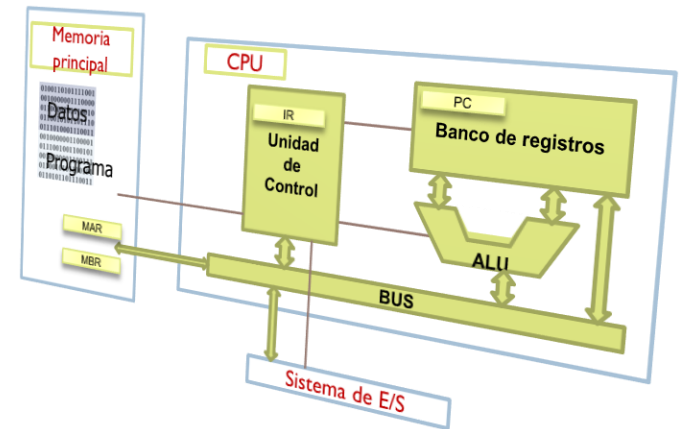
```
i = 0;
n = 45;  # número
s = 0;
while (i < 32)
{
    b = n & 1;
    s = s + b;
    n = n >> 1;
    i = i + 1 ;
}
```

```
li    t0, 0    #i
li    t1, 45   #n
li    t2, 32
li    t3, 0    #s
while: bge    t0, t2, fin
      andi    t4, t1, 1
      add     t3, t3, t4
      srli    t1, t1, 1
      addi    t0, t0, 1
      j       while
fin:   ...
```

Tipo de instrucciones

resumen

- ▶ Transferencias de datos
- ▶ Aritméticas
- ▶ Lógicas
- ▶ De desplazamiento
- ▶ De comparación
- ▶ Control de flujo (bifurcaciones, llamadas a procedimientos)
- ▶ De conversión
- ▶ De Entrada/salida
- ▶ Llamadas al sistema



Fallos típicos

1) Programa mal planteado

- ▶ No hace lo que se pide
- ▶ Hace incorrectamente lo que se pide

2) Programar directamente en ensamblador

- ▶ No codificar en pseudo-código el algoritmo a implementar

3) Escribir código ilegible

- ▶ No comentar el código ensamblador o no hacer referencia al algoritmo planteado inicialmente
- ▶ No tabular/estructurar el código para que sea legible por humano

Grupo ARCOS

uc3m | Universidad **Carlos III** de Madrid

Tema 3: Fundamentos de la programación en ensamblador (I) **Estructura de Computadores**

Grado en Ingeniería Informática
Grado en Matemática aplicada y Computación
Doble Grado en Ingeniería Informática y Administración de Empresas



Ejercicio

- ▶ Obtener los 16 bits superiores de un registro (t0) y dejarlos en los 16 bits inferiores de otro (t1)

Ejercicio (solución)

- Obtener los 16 bits superiores de un registro (t0) y dejarlos en los 16 bits inferiores de otro (t1)

```
srli    t1,    t0,    16
```



Se desplaza a la derecha 16
Posiciones (de forma lógica)

Ejercicio

- ▶ Determinar si el contenido de un registro (t2) es par.
Si es par se almacena en t1 un 1, sino se almacena un 0

Ejercicio (solución)

- Determinar si el contenido de un registro (t2) es par.
Si es par se almacena en t1 un 1, sino se almacena un 0

```
li    t2,  9
li    t1,  2
rem   t1,  t2,  t1    # se obtiene el resto
bne   t1,  zero,  else # cond.
then: li    t1,  1
      j     fin   # incond.
else:  li    t1,  0
fin:  ...
```

Ejercicio (otra solución)

- Determinar si el contenido de un registro (t2) es par.
Si es par se almacena en t1 un 1, sino se almacena un 0

```
li    t2,  9
li    t1,  2
rem   t3,  t2,  t1    # se obtiene el resto
li    t1,  0          # suponer impar
bne   t3,  zero,  fin  # si suposición ok, fin
li    t1,  1
fin:  ...
```

Ejercicio

- ▶ Determinar si el contenido de un registro (t2) es par. Si es par se almacena en t1 un 1, sino se almacena un 0. En este caso consultando el último bit

Ejercicio (solución)

- Determinar si el contenido de un registro (t2) es par. Si es par se almacena en t1 un 1, sino se almacena un 0. En este caso consultando el último bit

```
li    t2, 9
li    t1, 1
and   t1, t2, t1      # se obtiene el último bit
beq   t1, zero  then  # cond.
else: li    t1, 0
      j     fin      # incond.
then: li    t1, 1
fin:  ...
```

Ejercicio

- ▶ Determinar si el contenido de un registro (t2) es par. Si es par se almacena en t1 un 1, sino se almacena un 0. Sin utilizar instrucciones de bifurcación

Ejercicio (solución)

- Determinar si el contenido de un registro (t2) es par. Si es par se almacena en t1 un 1, sino se almacena un 0. Sin utilizar instrucciones de bifurcación

```
li    t2, 9
li    t1, 1
and   t1, t2, t1 # se obtiene el último bit
                # Si es par → t1 = 0
                # si es impar → t1 = 1
                # hay que cambiar el valor del bit
xori  t1, t1, 1
```

Ejercicio

- ▶ Calcular a^n
 - ▶ a en t0
 - ▶ n en t1
 - ▶ El resultado en a0

```
a=8
n=4;
i=0;
p = 1;
while (i < n)
{
    p = p * a
    i = i + 1 ;
}
}
```

Ejercicio (solución)

- ▶ Calcular a^n
 - ▶ a en t0
 - ▶ n en t1
 - ▶ El resultado en a0

```
a=8
n=4;
i=0;
p = 1;
while (i < n)
{
    p = p * a
    i = i + 1 ;
}
}
```

```
li    t0, 8
li    t1, 4
li    t2, 1
li    t4, 0

while: bge    t4, t1, fin
      mul    t2, t2, t0
      addi   t4, t4, 1
      j      while
fin:   mv     a0, t2
```

Ejemplo

- Calcular el número de 1's que hay en un `int` en C/Java

Otra solución :

```
int count[256] = {0,1,1,2,1,2,2,3,1, . . . 8};  
int i;  
int c = 0;  
  
for (i = 0; i <4; i++) {  
    c = count[n & 0xFF];  
    s = s + c;  
    n = n >> 8;  
}  
printf("Hay %d\n", c);
```

Ejercicio

Dada la siguiente expresión de un lenguaje de alto nivel

```
int a = 6;
```

```
int b = 7;
```

```
int c = 3;
```

```
int d;
```

```
d = (a+b) * (a+b);
```

Indique un fragmento de código en ensamblador del RISC-V 32 que permita evaluar la expresión anterior. El resultado ha de almacenarse en el registro t5.